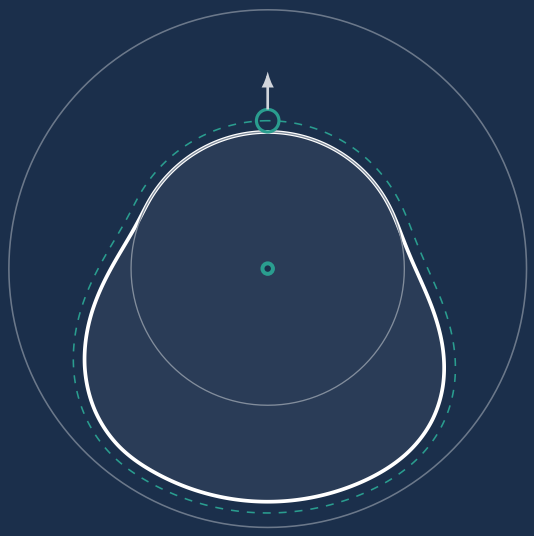
ELECTRONIC CAMMING ON THE TEKNIC CLEARCORE

Electronic CAM Workbench

Math and Algorithms

The formulas and algorithms of the Workbench and the ClearCore firmware, each with the code that implements it



Software version 0.11.1 | ECW-MAR-100, Revision A

AUTHOR AND DEVELOPER

Adam G. Sweeney

agsweeney@gmail.com

Companion document: *Cam Law Mathematics and ClearCore Theory of Operation*, ECW-TOP-100.

November 2026

MIT License

Electronic CAM Workbench: Math and Algorithms
Document ECW-MAR-100, Revision A, November 2026
For Electronic CAM Workbench and ECam firmware version 0.11.1

Copyright © 2026 Adam G. Sweeney. All rights reserved.

The Electronic CAM Workbench and its ECam firmware are released under the MIT License. The full license text is in the file `LICENSE` at the root of the source repository. The software is supplied "as is," without warranty of any kind.

Teknic®, ClearCore™, and ClearPath® are trademarks of Teknic, Inc. All other product names are trademarks of their respective owners. Teknic, Inc. does not publish or support this document.

Source of this document. This is `docs/MATH.md` of the source repository, typeset. Every formula is the one in the code, and each section names the file and the function that implements it. The charts are calculated by that code from the example projects in `samples/`.

Revision	Change
A, November 2026	First issue, for software version 0.11.1.

Contents

Figures **v**

Tables **v**

Preface **ix**

1 Notation and the Motion Laws **1**
1.1 Notation . . . 1
1.2 The Motion Laws . . . 1
1.3 A Segment in Real Units . . . 3

2 Blend and the Cubic Spline **5**
2.1 Blend: Matching the Neighbours . . . 5
2.2 Cubic Spline: A Curve Through Points . . . 6

3 The Profile, Analysis, and Speed Limits **9**
3.1 The Profile: Knots, Dwells, Closed and Open . . . 9
3.2 Analysis . . . 9
3.3 Speed Limits . . . 10

4 Plate-Cam Geometry **11**

5 Mechanics and the Runtime on the ClearCore **13**
5.1 Mechanics and Exact Gear Ratios . . . 13
5.2 The Runtime . . . 13

6 The Electronic Gear and Checksums **17**
6.1 The Electronic Gear . . . 17
6.2 Hashes and Checksums . . . 17

Bibliography **19**

This page is intentionally left blank.

List of Figures

1.1 The rest-to-rest laws: position, velocity, and acceleration over one segment. 2

2.1 Dwell, Blend up to line speed, Linear sync, Blend down: the carriage of the true flying cutoff. 6

4.1 The pressure angle of a translating radial roller follower: the carton pusher at 110° on its rise. Solid: the cam. Dashed: the pitch curve. Thin: the base circle. 11

This page is intentionally left blank.

List of Tables

1.1 Symbols 1
1.2 The pieces of S'' before scaling 3
1.3 Peak coefficients of the rest-to-rest laws 3

2.1 The Blend basis 5
2.2 The end conditions of a spline run 7

5.1 Counts per unit for each drive 13
5.2 Fixed-point values of the runtime 14

This page is intentionally left blank.

Preface

Purpose of This Document

This document is the reference for the mathematics and the algorithms of the Electronic CAM Workbench and its ECam firmware on the Teknic ClearCore. It shows how a profile becomes motion: the motion laws, the Blend and cubic-spline laws, the analysis and the speed limits, the plate-cam geometry, and the integer runtime that the ClearCore runs. It is short and close to the code. Each formula is the one that the code calculates, and each section names the file and the function that implement it.

The companion document, *Cam Law Mathematics and ClearCore Theory of Operation* (ECW-TOP-100), explains the same mathematics at more length, with worked examples. The user manual (ECW-UM-100) tells you how to use the Workbench.

Audience

This document is for engineers who change or check the Workbench and its firmware, and for engineers who must know exactly what the Workbench calculates. You must know calculus and the terms of motion control.

Conventions

Chapter 1 gives the symbols. A function or a file in the code is set as `EvaluateSegment` or `core/CamProfile.cpp`. A prime is a derivative with respect to the master (per degree on a crank, per second of the diagram on a timeline). Equations, figures, and tables are numbered by chapter.

> **NOTE:** If this document and the code are different, the code is correct. Use the code, and report the difference.

This page is intentionally left blank.

Chapter 1

Notation and the Motion Laws

1.1 Notation

Table 1.1. Symbols

Symbol	Meaning
x	Master position: the crank angle in degrees (0 to 360) or the timeline time in seconds
$s(x)$	Follower position, in the unit of the axis (mm, deg, ...)
s', s'', s'''	Derivatives with respect to the master: unit/deg, unit/deg^2, unit/deg^3 (per second on a timeline)
u	Fraction of a segment, $0 \le u \le 1$
ω	Master rate: deg/s on the crank (6× rpm), the speed override on the timeline
α	Angular acceleration of the crank while a driven crank ramps, deg/s^2
V, A	Velocity and acceleration limits of an axis, unit/s and unit/s^2

The derivatives with respect to the master become values per second by the chain rule, at a constant master speed (`MasterRate`, `DegPerSecond` in `core/CamProfile.h`):

$$\text{velocity} = s'\,\omega, \qquad \text{acceleration} = s''\,\omega^2, \qquad \text{jerk} = s'''\,\omega^3. \tag{1.1}$$

While the master accelerates there is one more term (section 3.3).

1.2 The Motion Laws

Each law is a normalized function $S(u)$ with its derivatives S', S'', S''' with respect to u (`EvaluateLaw`, `core/CamLaws.cpp`). A rise law has $S(0) = 0$ and $S(1) = 1$. A dwell has $S = 0$: its end position is tied to its start (section 3.1).

1.2.1 Sinusoidal-Acceleration Laws

The simple harmonic, cycloidal, modified trapezoid, and modified sine laws are built from pieces. The acceleration of each piece is a constant plus a sine. A piece starts at u_a:

$$S''(u) = c + k\sin(wt + \varphi), \qquad t = u - u_a. \tag{1.2}$$

Each piece is integrated twice in closed form. The velocity and the position continue from the end of the piece before:

$$S'(u) = S'(u_a) + c\,t + \frac{k}{w}(\cos\varphi - \cos(wt + \varphi)), \tag{1.3}$$

$$S(u) = S(u_a) + S'(u_a)\,t + \tfrac{1}{2}c\,t^2 + \frac{k}{w}\,t\cos\varphi - \frac{k}{w^2}(\sin(wt + \varphi) - \sin\varphi). \tag{1.4}$$

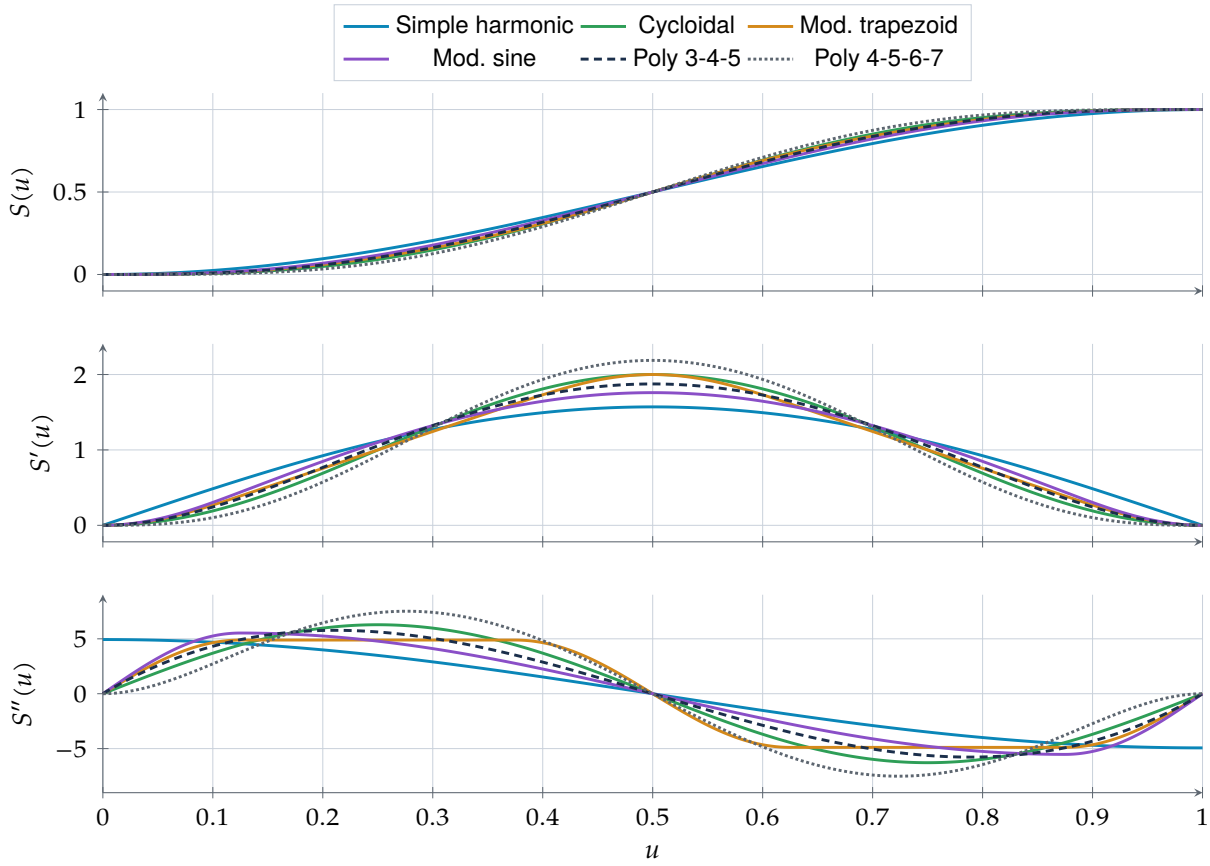


Figure 1.1. The rest-to-rest laws: position, velocity, and acceleration over one segment.

A piece with $w = 0$ has a constant acceleration. The full law is then divided by $S(1)$, so the rise is exactly 1 (`Build`, `EvalPieceLaw`).

Table 1.2. The pieces of S'' before scaling

Law	Pieces of S''
Simple harmonic	$\cos \pi u$ on $0 \le u \le 1$
Cycloidal	$\sin 2\pi u$ on $0 \le u \le 1$
Modified trapezoid	$\sin 4\pi u$ to $\frac{1}{8}$; 1 to $\frac{3}{8}$; $\cos 4\pi(u - \frac{3}{8})$ to $\frac{5}{8}$; -1 to $\frac{7}{8}$; $-\cos 4\pi(u - \frac{7}{8})$ to 1
Modified sine	$\sin 4\pi u$ to $\frac{1}{8}$; $\cos \frac{4\pi}{3}(u - \frac{1}{8})$ to $\frac{7}{8}$; $-\cos 4\pi(u - \frac{7}{8})$ to 1

1.2.2 Polynomial, Linear, and Dwell Laws

$$\text{3-4-5:} \quad S(u) = 10u^3 - 15u^4 + 6u^5, \tag{1.5}$$

$$\text{4-5-6-7:} \quad S(u) = 35u^4 - 84u^5 + 70u^6 - 20u^7. \tag{1.6}$$

The 3-4-5 law has zero velocity and acceleration at both ends. The 4-5-6-7 law also has zero jerk at both ends. **Linear** is $S = u$, $S' = 1$. **Dwell** is $S = 0$.

1.2.3 Peak Coefficients

The peak coefficients $C_v = \max|S'|$ and $C_a = \max|S''|$ are the values that `tests/test_camlaws.cpp` checks against the published values,[1] together with finite-difference checks of each derivative. $C_j = \max|S'''|$ comes from the code, sampled at 200,001 points.

Table 1.3. Peak coefficients of the rest-to-rest laws

Law	C_v	C_a	C_j	At the ends
Simple harmonic	1.5708	4.9348	15.503	acceleration steps
Cycloidal	2.0000	6.2832	39.478	zero acceleration
Modified trapezoid	2.0000	4.8881	61.426	zero acceleration, lowest C_a
Modified sine	1.7596	5.5280	69.466	zero acceleration, low C_v
Polynomial 3-4-5	1.8750	5.7735	60.000	zero acceleration
Polynomial 4-5-6-7	2.1875	7.5132	52.500	zero acceleration and jerk

1.3 A Segment in Real Units

A segment from (x_0, p_0) to (x_1, p_1) has the span $w = x_1 - x_0$ and the rise $h = p_1 - p_0$. With $u = (x - x_0)/w$ (`EvaluateSegment`, `core/CamProfile.cpp`):

$$s = p_0 + h\,S(u), \qquad s' = \frac{h}{w}\,S'(u), \qquad s'' = \frac{h}{w^2}\,S''(u), \qquad s''' = \frac{h}{w^3}\,S'''(u). \tag{1.7}$$

Thus, for a given law, the peak velocity increases with h/w and the peak acceleration with h/w^2: half the span gives four times the acceleration. The cubic spline is the exception: it is calculated directly in master units (section 2.2).

[1] Robert L. Norton, *Cam Design and Manufacturing Handbook*, 2nd ed. (New York: Industrial Press, 2009).

This page is intentionally left blank.

Chapter 2

Blend and the Cubic Spline

2.1 Blend: Matching the Neighbours

All the laws above except Linear start and end at rest. A segment that must *reach a speed*, for example a carriage that catches up with a moving line, needs a different law: Linear jumps to its speed at once, which is a velocity step. **Blend** starts at the velocity where the previous segment ends and finishes at the velocity where the next one starts, with zero acceleration at both ends (`EvaluateSegment`, `BlendNeighbourVel`).

It is the quintic Hermite curve with six end conditions: position $p_0 \to p_0 + h$, velocity $v_0 \to v_1$, and acceleration $0 \to 0$. With the end velocities per unit of u, $\hat{v}_0 = v_0 w$ and $\hat{v}_1 = v_1 w$:

$$s(u) = p_0 + h\,P(u) + \hat{v}_0\,B_0(u) + \hat{v}_1\,B_1(u), \tag{2.1}$$

The derivatives are taken term by term and divided by w, w^2, w^3, as in section 1.3.

Table 2.1. The Blend basis

Term	Polynomial	Role
$P(u)$	$10u^3 - 15u^4 + 6u^5$	the rise: $P(1) = 1$, flat at both ends
$B_0(u)$	$u - 6u^3 + 8u^4 - 3u^5$	the start velocity: $B_0'(0) = 1$, all else 0 at both ends
$B_1(u)$	$-4u^3 + 7u^4 - 3u^5$	the end velocity: $B_1'(1) = 1$, all else 0 at both ends

2.1.1 The Neighbours

v_0 is the velocity at the end of the previous segment, and v_1 the velocity at the start of the next. On a crank the profile wraps: the segment before the first is the last. The advance of an open profile is only an offset, so its velocity wraps too. Before the start and after the end of a timeline axis, the axis is at rest. A neighbour that is also a Blend counts as at rest, so Blends never depend on each other. An axis with one segment has no neighbours. Between two rests ($v_0 = v_1 = 0$), Blend is exactly the 3-4-5 polynomial.

2.1.2 The Natural Rise

For a Blend from rest to a speed v, any rise works, but $h = \frac{1}{2}vw$ (the distance covered at the average speed) gives the cleanest motion:

$$s(u) = vw\left(u^3 - \tfrac{1}{2}u^4\right), \qquad s' = v\left(3u^2 - 2u^3\right), \qquad s'' = \frac{6v}{w}\,u(1-u). \tag{2.2}$$

The velocity increases monotonically from 0 to v (a smoothstep), the acceleration is one positive hump with the peak $1.5\,v/w$, and nothing overshoots. A larger rise makes the velocity overshoot v and come back. A smaller rise undershoots.

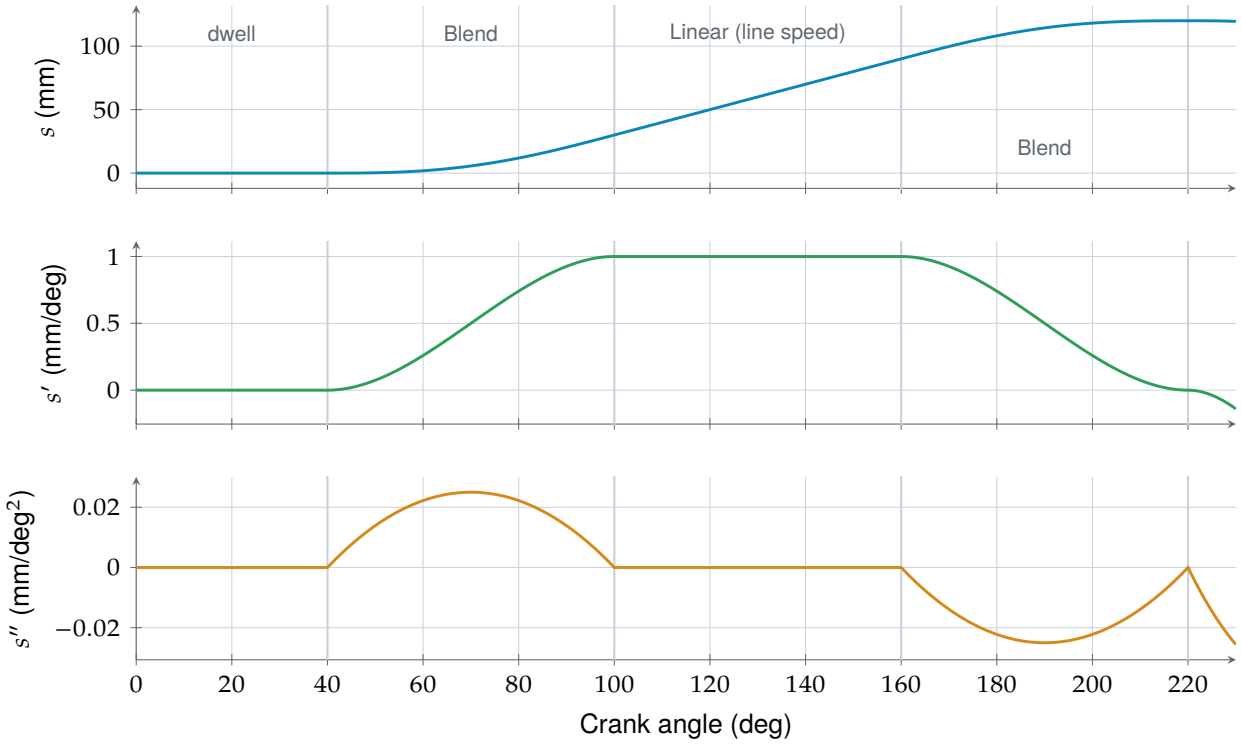


Figure 2.1. Dwell, Blend up to line speed, Linear sync, Blend down: the carriage of the true flying cutoff.

Example. In `samples/true_flying_cutoff.ecam` the line moves 1 mm for each degree of the crank. The carriage dwells to 40°, blends from 40° to 100° ($w = 60°$, $h = 30\,\text{mm} = \frac{1}{2} \cdot 1 \cdot 60$), rides the line from 100° to 160° on a Linear segment ($s' = 1\,\text{mm/deg}$ exactly), and blends from 160° to 220° back to rest. The peak acceleration in the blends is $1.5 \cdot 1/60 = 0.025\,\text{mm/deg}^2$. At 8 rpm ($\omega = 48\,°\,\text{s}^{-1}$) that is $0.025 \cdot 48^2 = 57.6\,\text{mm}\,\text{s}^{-2}$. `test_camlaws` checks that this profile has no velocity or acceleration step, that the sync is at line speed, and the derivatives against finite differences.

2.2 Cubic Spline: A Curve Through Points

Each other law is a closed form between two knots. The **cubic spline** is for a shape given by points. A run of consecutive spline segments is one curve through all their knots: a cubic on each piece, with the position, velocity, and acceleration continuous at each knot inside the run (`FitSplines`, `EvaluateSegment`). The method is the one in the lecture notes of Ruye Wang.[1]

2.2.1 The Piece

The unknowns are the second derivatives M_i at the knots. Between the knots x_{i-1} and x_i, with $h_i = x_i - x_{i-1}$, the second derivative is a straight line from M_{i-1} to M_i. Two integrations, with

[1] Ruye Wang, "Spline Interpolation," lecture notes, Harvey Mudd College, `https://pages.hmc.edu/ruye/MachineLearning/lectures/ch7/node6.html`.

the knot positions as the constants, give:

$$C(x) = M_{i-1}\frac{(x_i - x)^3}{6h_i} + M_i\frac{(x - x_{i-1})^3}{6h_i} + \left(\frac{y_{i-1}}{h_i} - \frac{M_{i-1}h_i}{6}\right)(x_i - x) + \left(\frac{y_i}{h_i} - \frac{M_i h_i}{6}\right)(x - x_{i-1}). \tag{2.3}$$

The position and the acceleration now match at each knot by construction. The jerk is constant on each piece: $(M_i - M_{i-1})/h_i$.

2.2.2 The Equations

The velocity must match at each inner knot. With $\mu_i = h_i/(h_i + h_{i+1})$ and $\lambda_i = 1 - \mu_i$:

$$\mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = 6f[x_{i-1}, x_i, x_{i+1}], \qquad f[x_{i-1}, x_i, x_{i+1}] = \frac{f[x_i, x_{i+1}] - f[x_{i-1}, x_i]}{x_{i+1} - x_{i-1}}. \tag{2.4}$$

The matrix is tridiagonal and strictly diagonally dominant, so the Thomas algorithm solves it without pivoting, in a time proportional to the number of knots. Only the slopes $f[x_{i-1}, x_i]$ occur, so the advance of an open profile across the wrap needs no special handling.

2.2.3 The Ends

Two more equations are necessary. The neighbours of the run decide them: The right end mirrors

Table 2.2. The end conditions of a spline run

Next to the run	End	Row of the matrix
Any other law (a dwell has velocity 0)	Clamped to its velocity v	$2M_0 + M_1 = \frac{6}{h_1}(f[x_0, x_1] - v)$
The start or end of a timeline axis	Clamped to 0	the same, $v = 0$
A Blend	Natural	$M_0 = 0$
Nothing: each segment of a cycle is a spline	Periodic	the equation above, wrapped

the left: $M_{n-1} + 2M_n = (6/h_n)(v - f[x_{n-1}, x_n])$.

A clamped end keeps the velocity continuous but not the acceleration, so a spline that leaves a dwell has an acceleration step, which the analysis reports. The natural end makes a Blend the smooth exit: a Blend has zero acceleration at its ends, the natural end has $M = 0$, and the Blend takes the velocity of the spline, so the join is continuous in position, velocity, and acceleration. A spline next to a Blend never reads the velocity of the Blend, which keeps the two from depending on each other.

2.2.4 The Periodic Spline

When the knot after the last is the first, the matrix gets two corner entries: $\beta = \mu_0$ at the top right and $\alpha = \lambda_{n-1}$ at the bottom left. The Sherman–Morrison formula removes them.[2] With $\gamma = -2$,

[2] Jack Sherman and Winifred J. Morrison, "Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix," *Annals of Mathematical Statistics* 21, no. 1 (1950): 124–27.

let T be the tridiagonal part with γ subtracted from its first diagonal entry and $\alpha\beta/\gamma$ from its last. Solve $Ty = d$ and $Tz = q$ for $q = (\gamma, 0, \dots, 0, \alpha)$. Then

$$M = y - z\,\frac{y_0 + \beta\, y_{n-1}/\gamma}{1 + z_0 + \beta\, z_{n-1}/\gamma}. \tag{2.5}$$

Two knots are solved directly. One piece around the cycle is a straight line.

2.2.5 Splitting, Overshoot, and the Board

Splitting a segment adds a knot on the curve. The spline through the same points with the same ends is unique, so the new spline is identical, and `SplitAt` changes nothing until you move the knot.

Overshoot. A spline does not know the range that you drew, and it swings past its points where they change direction sharply. `Analyze` reports `SplineOvershoot` when a spline segment passes the highest or the lowest knot of the axis by more than 2 % of their range.

On the board nothing new runs in the sample interrupt. The table of the runtime samples the curve as it samples any other law (chapter 5), solved once for each axis (`SplineFit`). A cubic in the interrupt would gain nothing, because the linear interpolation between table entries is in error by at most $h^2|s''|/8$: 0.03 counts for a 20 mm rise at 800 counts/mm in 120°, against the ± 0.5 count of the rounding of the table.

`test_camlaws` checks: a cubic reproduced exactly on uneven knots; the equations above on the solved M at each knot; a natural case worked by hand (knots $(0,0)$, $(1,1)$, $(2,0)$: $M_1 = -3$, end slope 1.5); a periodic spline through the wrap and within 2×10^{-5} of a 24-knot sine; the derivatives against finite differences; and splitting.

CHAPTER 3

The Profile, Analysis, and Speed Limits

3.1 The Profile: Knots, Dwells, Closed and Open

An axis is a list of segments over its master span: 360° on the crank, the duration on the timeline. The boundaries of the segments are the **knots** (`core/CamProfile.cpp`).

- **Dwells are tied.** The end knot of a dwell always has the position of its start knot. The knots are grouped by these ties (a small union-find, `KnotGroups`), and each knot takes the value of the knot with the lowest index in its group. Thus a drag of the knot before a dwell moves the full dwell.
- **Closed** profiles tie the last knot to knot 0: the follower is back where it started each revolution. **Open** profiles do not: they advance by $\Delta = s_{\text{end}} - s_{\text{start}}$ each revolution or cycle (an index feed).
- **Cyclic.** A crank axis wraps from 360° directly into 0°. A repeating timeline axis that is as long as the cycle does too. The other timeline axes rest at their start before 0 and at their end after their duration. The step checks (section 3.2) include the wrap on cyclic axes, and the rest at the start and the end on the others.
- **A change of master** keeps the shape. Crank to timeline scales the segment ends so that one revolution at the current speed becomes seconds, $T = 60/\text{rpm}$. Timeline to crank scales the duration to 360° (`SetAxisMaster`, `ScaleSpan`).
- The shortest segment is 0.5° on a 360° crank, scaled with the span on a timeline.

3.2 Analysis

`Analyze` (`core/CamProfile.cpp`) samples the axis at 3600 points for each span (1440 in the speed-limit functions) and reports these values to the panel in the window (`MainWindow::RefreshStats`):

- **Stroke and peaks:** the minimum and maximum position, and the peaks of $|s'|$, $|s''|$, $|s'''|$ with where they occur, shown per second as $|s'|\omega$, $|s''|\omega^2$, $|s'''|\omega^3$.
- **Motor**, with c the motor counts per unit and N the master counts for each crank revolution:

$$\begin{aligned} \text{peak step rate} &= \max|s'|\,\omega\,c && \text{counts/s,} \\ \text{peak motor speed} &= 60 \cdot \text{peak step rate}/\text{ppr} && \text{rpm,} \\ \text{ratio (crank)} &= \max|s'|\,c \cdot 360/N && \text{motor counts per master count.} \end{aligned}$$

- **Steps at the knots.** At each knot, the one-sided values of the segment that ends there (left) and the segment that starts there (right) are compared. On a cyclic axis this includes the

wrap. On a timeline, the axis is at rest before its start and after its end:

$$\begin{aligned} \text{velocity step:} \quad & |s'_L - s'_R| > 10^{-6} \max(|s'_L|, |s'_R|) + 10^{-9}, \\ \text{acceleration step:} \quad & |s''_L - s''_R| > 10^{-6} \max(|s''_L|, |s''_R|) + 10^{-12}. \end{aligned}$$

A velocity step means an infinite acceleration (an error). Otherwise an acceleration step means an infinite jerk (a warning). The displacement diagram marks each problem with a warning sign over the curve.

- **Cam drawing:** the pressure angle and undercut (chapter 4).

3.3 Speed Limits

Crank axes. At the crank speed ω and the crank acceleration α, the acceleration of the follower is

$$\text{acceleration} = s''\omega^2 + s'\alpha. \tag{3.1}$$

The second term is the follower pulled faster as the crank speeds up. With the limits V and A, the fastest safe crank speed is the smallest value, over the full profile, of

$$\omega \le \frac{V}{|s'|}, \qquad \omega \le \sqrt{\frac{A - |s'|\alpha}{|s''|}}, \tag{3.2}$$

with both terms at their worst sign. If $|s'|\alpha \ge A$ anywhere, no speed is safe: the ramp itself is too steep. The steepest ramp that the profile accepts is $\alpha_{\max} = A / \max|s'|$ (`MaxMasterSpeed`, `core/CamProfile.cpp`). α is the ramp of the driven crank (6× its rpm/s). The Workbench cannot ramp an encoder master, so $\alpha = 0$ there (`RampDegPerSecond2`). The Crank panel shows the lowest limit over all crank axes (and the maximum of a driven crank motor), green when the set speed is below it (`MainWindow::ProjectSpeedLimit`).

Timeline axes. The master rate is the speed override (seconds of the diagram per second), with no ramp term: the timeline starts at speed because each axis starts from rest. The same formulas with $\alpha = 0$ give the highest safe override (`MainWindow::TimelineOverrideLimit`).

Chapter 4

Plate-Cam Geometry

The cam view and the DXF draw each crank axis as a plate cam with a translating radial roller follower at 12 o'clock. The cam turns clockwise. These drawings do not change the motion: the base radius R_b and the roller radius r_r are drawing settings.

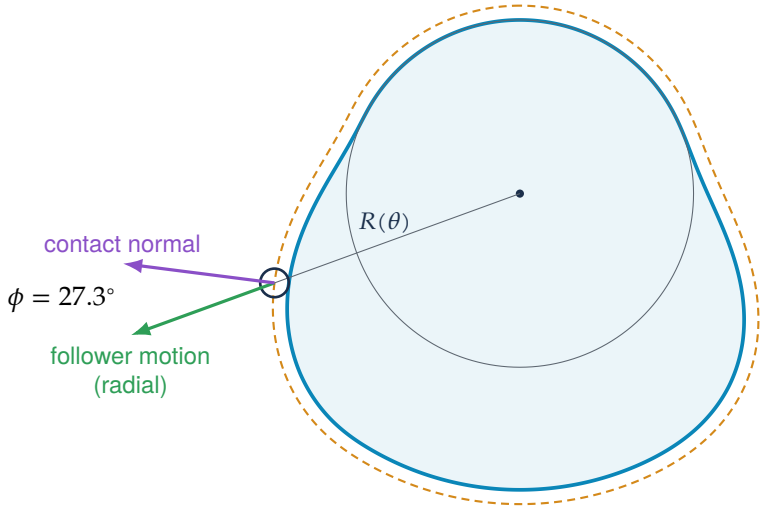


Figure 4.1. The pressure angle of a translating radial roller follower: the carton pusher at 110° on its rise. Solid: the cam. Dashed: the pitch curve. Thin: the base circle.

Pitch curve. The pitch curve is the path of the center of the roller (`PitchPointAndNormal`, `core/CamGeometry.cpp`). In the frame of the cam the follower is at the angle $90° + \theta$, at the radius

$$R(\theta) = R_b + r_r + s(\theta) - s_{\min}. \tag{4.1}$$

Pressure angle. The pressure angle ϕ (`Analyze`) is the angle between the motion of the follower (radial) and the contact normal:

$$\tan\phi = \frac{|R'|}{R}, \qquad R' = \frac{dR}{d\theta} \text{ per radian} = s' \frac{180}{\pi}. \tag{4.2}$$

Above 30° the follower is pushed sideways more than along its guide, and the analysis gives a warning.

Radius of curvature. For the pitch curve as a polar curve $R(\theta)$, with $R'' = s''(180/\pi)^2$:

$$\rho = \frac{\left(R^2 + R'^2\right)^{3/2}}{R^2 + 2R'^2 - R\,R''}. \tag{4.3}$$

Where the denominator is positive, the pitch curve is convex. **Undercut**: where a convex part has $\rho < r_r$, the roller is larger than the curve that it must follow, and it would cut its own path. The analysis gives a warning.

The cam outline. The outline is the envelope of the roller (`BuildCamShape`). For each ray from the cam center, with the unit vector $\hat{u}$, the surface is where the ray first enters a roller circle centered on the pitch curve. The cam is everything that the roller never reaches. For a pitch point P:

$$a = P \cdot \hat{u} \ \ (\text{along the ray}), \qquad b = |P \times \hat{u}| \ \ (\text{across it}), \tag{4.4}$$

and the ray meets that roller at $t = a - \sqrt{r_r^2 - b^2}$ when $a > 0$ and $b \le r_r$. The surface radius on that ray is the smallest t over the pitch points that can reach it. A roller at a radius of at least $R_b + r_r$ spans at most $\arcsin(r_r/(R_b + r_r))$ on each side of its center, so only that window of samples is searched. Where the profile is smooth, this is the pitch curve moved by r_r along its normal. At a velocity step (a corner in the pitch curve) it trims the overlap of an inside corner and rounds an outside corner. At the drop of an open profile it cuts the relief that the roller rolls off by. The DXF writes this outline with 360 to 36,000 points for each revolution.

Chapter 5

Mechanics and the Runtime on the ClearCore

5.1 Mechanics and Exact Gear Ratios

Counts per unit from the drive (`DriveCountsPerUnit`):

Table 5.1. Counts per unit for each drive

Drive	Counts per unit
Direct	c as entered
Ball screw	$c = \text{ppr}/\text{pitch}$
Belt	$c = \text{ppr}/(\pi \cdot \text{pulley pitch diameter})$

Master counts. With a gear ratio $n : d$ (motor or encoder turns to crank turns), the crank has $N = \text{ppr} \cdot n/d$ counts for each revolution. N need not be whole (800 ppr at 50 : 7 is 5714.29), so the board never counts in counts. It counts in **units of** $1/d$ **of a count**, with $M = \text{ppr} \cdot n$ units for each crank revolution, a whole number (`CrankUnitsPerRev`). Each encoder count or crank-motor step is d units.

At 800 ppr and 50 : 7: $M = 40{,}000$ units for each crank turn and a step is 7 units, so each 7 crank turns are exactly $7 \cdot 40{,}000/7 = 40{,}000$ steps. The 0° of the crank never creeps, however long it runs. M is at most 2×10^9 (`kMaxCrankUnits`).

5.2 The Runtime

`core/CamRuntime.cpp` runs on the board in the 5 kHz sample interrupt, and on the PC in `tests/test_runtime.cpp`. `Build` does the floating-point work once, in the main loop, when a profile is loaded. `Tick`, each 200 µs, uses whole numbers only.

5.2.1 Tables

Each axis is sampled at $K + 1 = 2049$ equally spaced master positions into motor counts, in double precision. The last entry comes from the end of the last segment:

$$T[k] = \text{round}(s(k \cdot \text{span}/K) \cdot c), \qquad k = 0, \ldots, 2048. \tag{5.1}$$

Between entries the runtime interpolates linearly, with a 16-bit fraction f (0 to 65535):

$$\text{target} = T[i] + ((T[i+1] - T[i]) \cdot f) \gg 16. \tag{5.2}$$

The arithmetic shift rounds down. The interpolation error is at most $\frac{1}{8}\,\Delta x^2 \max|s''|\,c$ counts with $\Delta x = \text{span}/2048$. On a crank $\Delta x = 0.176°$, so for $|s''| = 0.025\,\text{mm/deg}^2$ and $c = 160$ counts/mm it is 0.015 counts: well below a step.

5.2.2 Fixed Point

"Q24" and "Q16" mean a value multiplied by 2^{24} or 2^{16} and kept as a whole number (a binary fraction). The crank phase stays below $M \cdot 2^{24} \le 2 \times 10^9 \cdot 2^{24} \approx 3.4 \times 10^{16}$, far inside an int64.

Table 5.2. Fixed-point values of the runtime

Quantity	Representation
Crank phase	master units within the revolution, Q24, in an int64, plus a revolution count (int32)
Crank velocity	master units per sample, Q24
Timeline time	samples into the cycle, Q16, plus a cycle count
Timeline rate	Q16; 65536 is a speed override of 1.0

Whole turns go into the revolution count, so a long run never overflows. With φ the phase in whole units and D the cycle as drawn ($D = M$ except in cut-length mode), the table index and the fraction are

$$i = \left\lfloor \frac{\varphi K}{D} \right\rfloor, \qquad f = \frac{(\varphi K \bmod D) \ll 16}{D}. \tag{5.3}$$

On a driven crank or a jog, the fraction of a unit in the phase is added to f. The timeline divides by the span of the axis in samples in the same way.

5.2.3 The Crank Master

- **Encoder:** each sample the phase moves by the change of the encoder times d ($\Delta\text{phase} = \Delta\text{counts} \cdot d$, in Q24). The wrap of the 32-bit counter has no effect because the change is taken in 32 bits. The callback sees the encoder reading of the previous sample.
- **Driven crank:** the velocity moves toward its target by at most the ramp in each sample, both in Q24:

$$\begin{aligned} \text{target velocity} &= \text{rpm} \cdot M/60/5000 && \text{units per sample,} \\ \text{ramp} &= (\text{rpm/s}) \cdot M/60/5000^2 && \text{units per sample, per sample.} \end{aligned}$$

  The crank motor gets $\lfloor \text{units}/d \rfloor$ steps in total (the floor, also below zero), sent as the change in each sample.

5.2.4 Cut-Length Mode

The encoder measures the line at q counts/mm (encoder counts per revolution divided by the wheel circumference). The profile is drawn over a design cycle of D counts ($D = \text{design length} \cdot q$, rounded). The cycle actually cut is C counts, which can change:

$$C = \text{round}((\text{piece} + \text{kerf})\, q), \qquad x = \max(0,\ \text{phase} - (C - D)). \tag{5.4}$$

All look-ups use x instead of the phase (axis tables, crank output pulses). The drawn cycle starts with a wait W (design counts) in which each crank axis dwells, and x stays inside the wait while phase $< C - D + W$. A longer cycle ($C > D$) holds x at 0 longer. A shorter cycle ($C < D$) starts x part of the way into the wait. After the wait, x and the phase advance together, so the accelerate, sync, decelerate, and return part keeps its line distance: the speeds and accelerations of the carriage at a given line speed do not depend on the length, and neither do the limits. The shortest cycle is $D - W$ (no wait), so the shortest piece is $(D - W)/q - \text{kerf}$.

The cut line. The sync is the first Linear segment of the carriage, from S_0 to S_1 (design counts). A point of the material passes the home position of the blade and the blade meets it when the carriage reaches the sync start: that point is the cut line. Its distance behind the blade, in line counts, is the line travel still to the next sync start, less the carriage travel still to its sync position:

$$N = C - D + S_0 \qquad \text{while phase} < C - D + S_1 \text{ (this cycle's sync)}, \tag{5.5}$$

$$N = C + (C_{\text{next}} - D + S_0) \qquad \text{after it (the next cycle)}, \tag{5.6}$$

$$\text{gap} = (N - \text{phase}) - (c(S_0) - c_{\text{now}})\, q/c_{\text{mm}}, \tag{5.7}$$

with c in carriage motor counts and c_{mm} the counts per millimetre of the carriage. In the sync the carriage moves with the line, so the gap is 0 there; the board reports it as on the cut line, with the line travel left before S_1. Before the sync the gap decreases at the line speed during the wait, and more slowly while the carriage accelerates. Consecutive cut lines are exactly C apart on the material, and a new length (taken at the wrap) applies to the next piece.

A new C is taken when the phase wraps, so the cut in progress finishes at its old length. While the crank is off, it is taken at once. While you jog by hand, it is taken when the jog ends (the jog targets are whole turns of the length that the jog started with). `test_runtime` checks a 600 mm cycle changed to 503 mm in the middle of a cut: the new length starts with the next cut, the cuts are exactly C counts apart, and the carriage stays within one count of its drawn profile at both lengths.

5.2.5 Timeline

Time advances by the rate in each sample. At the trigger the rate starts at the override (each axis starts from rest). A change of the override ramps to full in 0.25 s (65536/1250 per sample). A one-shot cycle ends at the cycle length (the longest timeline axis or the last timeline output pulse). A repeating cycle wraps and counts cycles. **Stop** is a cycle stop: a running timeline finishes its cycle and rests at its end. A second Stop ramps the rate to zero over `stop_ms`.

5.2.6 Steps Are Differences

In each sample, the motor of an engaged axis gets $\text{target}_{\text{now}} - \text{target}_{\text{last}}$ steps. Engaging sets $\text{target}_{\text{last}}$ to the current target first, so engaging never jumps: the motor position at that moment becomes the profile position. An open profile adds its advance $\Delta \cdot c$ once in each revolution or cycle.

5.2.7 Hand Jog

A hand jog moves the master toward a target with a velocity and an acceleration limit (`JogVelocity`). In each sample the wanted speed toward the target is

$$v = \min(v_{\max},\ \sqrt{2a|e|}), \qquad e = \text{distance left}, \tag{5.8}$$

approached by at most a in each sample. The last step lands on the target. The limits are the crank speed or the timeline speed of the profile, reached in a quarter of a second (a driven crank uses its ramp).

5.2.8 Outputs

The ends of the pulses are converted once at load: a crank pulse to master units, $\text{round}(x/360 \cdot M)$, and a timeline pulse to samples, Q16. In each sample an output is on when its master is engaged

or running and the phase or time is in [on, off). The pins are written only when the result changes (`CamEngine.cpp`), and never in simulation.

5.2.9 Steps to the Drive

The EnhancedClearCoreLibrary sends the steps (`MotorDriver::ApplySampleSteps`). The steps for a sample go out over the next 200 µs. Steps above the maximum of the step timer for one sample are carried into the next samples, not lost. The direction output changes only with no pulses in flight: a sample with no steps does not change it, and a reversal right after a sample that sent steps waits one sample (the steps are carried). Thus a drive that reads DIR slightly after the step edge never counts a step the wrong way. `test_runtime` checks the net steps and no drift over 20 revolutions.

Chapter 6

The Electronic Gear and Checksums

6.1 The Electronic Gear

An axis marked as a gear has no diagram: it moves `gearNum` units for each `gearDen` crank revolutions. In the model it is one open Linear segment of gearNum/gearDen units for each revolution, which is what a constant ratio is, so the diagrams, the analysis, and the speed limits need nothing new (`Normalize`).

The runtime cannot use that segment. The table holds whole counts, so the advance for each revolution would round: 3 : 7 of an 800-count motor is 342.857... counts for each revolution, and at 343 the axis would gain a count each 7 revolutions, without end. Thus a gear has no table and keeps a fraction instead. With U master units for each revolution (`CrankUnitsPerRev`):

$$\frac{P}{Q} = \frac{\text{countsPerUnit} \cdot \text{gearNum}}{\text{gearDen} \cdot U} \qquad \text{motor counts per master unit, whole numbers.} \tag{6.1}$$

In each sample, with Δ the motion of the master in whole units:

$$\text{acc} \leftarrow \text{acc} + \Delta P, \tag{6.2}$$

$$q = \lfloor \text{acc}/Q \rfloor, \qquad \text{acc} \leftarrow \text{acc} - qQ \qquad (0 \le \text{acc} < Q), \tag{6.3}$$

$$\text{steps} \leftarrow \text{steps} + q. \tag{6.4}$$

Nothing is rounded away: acc carries the remainder, so after any motion the position is exactly $\lfloor uP/Q \rfloor$ for the total motion u of the master, in either direction, and a return of the master returns the axis to the count. The floor keeps the remainder positive when the master goes backward, so forward and back are the same function of u.

Δ is the motion of the crank before its phase wraps, added up in Q24 wherever the phase moves: the change of the encoder, the velocity of the driven crank, a hand jog. Thus ramps, jogs, and stops need nothing of their own, and a gear engages and disengages with the crank axes. In cut-length mode U is the cycle as drawn, not the current cycle, so a gear keeps its ratio to the line while the cut length changes.

A `countsPerUnit` that is not whole (a belt: motor ppr/(πd)) has no exact fraction. It is doubled, with Q, until it is whole or until the numerator reaches 2^{37}, which keeps ΔP inside 64 bits for 2^{24} units in one sample and leaves an error below 1 part in 2^{38}. `Build` refuses a ratio whose numerator starts at 2^{38} or more, or whose Q starts above 4×10^{18}.

`test_runtime` compares each sample with the whole-number formula for seven ratios from 1 : 1000 to 997 : 991, also reversed, forward and back through zero, and with the count after 280,000 revolutions; it also checks a 50 : 7 crank, a belt, a driven crank through its ramps against its own motor, a hand jog, a restart, and cut-length mode with a changing cut.

6.2 Hashes and Checksums

- **Profile hash** (`ProfileHash`, `core/ProfileText.cpp`): the 32-bit FNV-1a hash of each upload line, each followed by a line feed:

$$h_0 = 2166136261, \qquad h \leftarrow (h \oplus b) \cdot 16777619 \pmod{2^{32}} \quad \text{for each byte } b.$$

The app and the board calculate it over the same lines (including the project copy), so equal hashes mean that the board runs exactly this project.

- **Flash store** (`Firmware/ECamFirmware/FlashStore.cpp`): the same FNV-1a over the stored text, in the header next to its length. At power-up the board ignores a stored profile whose checksum does not agree.
- **Settings** (`Settings.cpp`): FNV-1a over the settings block, with a magic word and a version.

Bibliography

Fowler, Glenn, Landon Curt Noll, and Kiem-Phong Vo. "FNV Hash." `http://www.isthe.com/chongo/tech/comp/fnv/`.

Norton, Robert L. *Cam Design and Manufacturing Handbook.* 2nd ed. New York: Industrial Press, 2009.

Sherman, Jack, and Winifred J. Morrison. "Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix." *Annals of Mathematical Statistics* 21, no. 1 (1950): 124–27.

Sweeney, Adam G. *Cam Law Mathematics and ClearCore Theory of Operation.* Document ECW-TOP-100, revision A. 2026.

———. *Electronic CAM Workbench User Manual.* Document ECW-UM-100, revision A. 2026.

Wang, Ruye. "Spline Interpolation." Lecture notes, Harvey Mudd College. `https://pages.hmc.edu/ruye/MachineLearning/lectures/ch7/node6.html`.